

Python

সিনিয়র লেভেল রিভিশন — ১৫ মিনিট। ফোকাস: backend/API কাজে Python (FastAPI-কেন্দ্রিক) + ভাষার সিনিয়র-লেভেল ধারণা।

১. GIL (Global Interpreter Lock) — আসল প্রভাব কী?

Description: Python-এর সবচেয়ে বিখ্যাত সীমাবদ্ধতা। "Python-এ threading অর্থহীন" — এই ভুল ধারণাটা শুধরে সঠিক ছবি দিতে পারা সিনিয়র উত্তর।

৬টা Key Point:

1. GIL: এক মুহুর্তে একটাই thread Python bytecode চালায় — CPU-bound কাজে multi-threading-এ লাভ নেই
2. কিন্তু I/O-তে GIL ছেড়ে দেয় — network/disk wait-এ অন্য thread চলে; I/O-bound কাজে threading দিব্যি কার্যকর
3. CPU-bound সমাধান: multiprocessing (আলাদা process, আলাদা GIL) বা C-extension (numpy/pandas ভেতরে GIL ছাড়ে)
4. সিদ্ধান্ত-টেবিল মুখস্থ: I/O-bound অনেক connection → asyncio; I/O-bound সীমিত → threads; CPU-bound → processes
5. Web জগতে বাস্তবতা: uvicorn/gunicorn একাধিক worker process চালায় — GIL কার্যত সমস্যা না, PHP-FPM/Node-cluster-এর মতোই
6. আপডেট থাকার টাচ: Python 3.13-এ experimental free-threaded (no-GIL) build এসেছে — মূলধারায় আসতে সময় লাগবে, এক লাইনে বলা যথেষ্ট

২. asyncio ও async/await Python-এ

Description: FastAPI-যুগে async Python জানা বাধ্যতামূলক — event loop মডেল আর কখন async def লিখবেন/লিখবেন না।

৬টা Key Point:

1. মডেল Node-এর মতোই: এক event loop, await-এ control ছাড়া — কিন্তু Python-এ sync জগৎটাও বিশাল, তাই মেশানোর নিয়ম জানা জরুরি
2. Async চেইন পুরোটা async হতে হয়: async route-এ sync blocking call (requests, time.sleep) দিলে পুরো loop জমে — সবচেয়ে কমন FastAPI বাগ
3. Blocking কাজ async জগতে: `run_in_executor/asyncio.to_thread` — থ্রেডে ঠেলে loop মুক্ত রাখা
4. FastAPI-র চালাকি: `def` route আপনিই threadpool-এ চলে, `async def` loop-এ — sync লাইব্রেরি হলে সংভাবে `def`-ই লেখা ভালো
5. Parallel I/O: `asyncio.gather(*tasks)` — একাধিক tenant/API একসাথে টানা (Sokrio benchmark-এ multi-tenant fan-out এভাবেই); timeout দিতে `asyncio.wait_for`
6. Async lib আলাদা: httpx (requests-এর বদলে), asyncpg/motor — driver sync হলে async-এর সব লাভ বাতিল

৩. FastAPI: আধুনিক Python API Framework

Description: FastAPI কেন এত জনপ্রিয় আর একটা প্রোডাকশন FastAPI সার্ভিস কীভাবে গঠন করবেন — দৈনন্দিন কাজের ফ্রেমওয়ার্ক-প্রশ্ন।

৬টা Key Point:

1. তিন স্তম্ভ: type hint থেকে validation (Pydantic) + auto OpenAPI docs + async native — boilerplate প্রায় শূন্য
2. Pydantic boundary-তে: request/response schema আলাদা model — ORM object সরাসরি ফেরানো নয়, `response_model` দিয়ে leak ঠেকানো
3. Dependency Injection (`Depends`): DB session, current user, tenant resolution — reusable, testable, route signature-এই দৃশ্যমান
4. Structure বড় হলে: APIRouter দিয়ে module ভাগ (router/service/schema আলাদা ফাইল) — সব main.py-তে নয় (Sokrio-python-এ এই split-ই)
5. Error handling: HTTPException + custom exception handler — এক জায়গায় consistent error envelope
6. Sync frameworks তুলনা এক লাইনে: Django (batteries-included, admin ফ্রি — Laravel-এর জাতভাই), Flask (minimal), FastAPI (API-first, async) — API সার্ভিসে FastAPI-ই এখন default পছন্দ

8. Decorator, Context Manager ও Generator

Description: Python-এর তিনটা signature idiom — এগুলো স্বচ্ছন্দে ব্যাখ্যা করতে পারা মানে ভাষাটা সত্যি জানেন।

৬টা Key Point:

1. Decorator = function নিয়ে wrapped function ফেরানো — cross-cutting কাজ (log, cache, retry, auth) মূল logic-এ হাত না দিয়ে
2. নিজের decorator-এ `functools.wraps` না দিলে নাম/docstring হারায় — ছোট কিন্তু অভিজ্ঞতা-প্রকাশক পয়েন্ট
3. প্রোডাকশন উদাহরণ বলা: `@lru_cache`, retry decorator, FastAPI-র route decorator নিজেই — theory নয়, ব্যবহার
4. Context manager (`with`) = নিশ্চিত cleanup — file/lock/DB transaction; exception হলেও `__exit__` চলে; `@contextmanager` + `yield` দিয়ে সহজে বানানো
5. Generator = lazy sequence: `yield` — বিশাল ডেটা এক item করে; মেমোরি ফ্ল্যাট (বড় query result/CSV-তে এটাই পথ)
6. Generator চেইন করা যায় (pipe-এর মতো): `read` → `filter` → `transform` প্রতিটা generator — Node stream-এর Python জবাব

৫. Python Data Structures ও Performance বাছাই

Description: dict/list/set-এর complexity জানা আর কোন কাজে কোনটা — ছোট প্রশ্ন মনে হলেও ভুল উত্তর বড় লাল পতাকা।

৬টা Key Point:

1. dict/set lookup $O(1)$ — "in list" $O(n)$; membership check-এ list ব্যবহার সবচেয়ে কমন performance ভুল
2. list append $O(1)$, কিন্তু সামনে insert $O(n)$ — queue লাগলে `collections.deque` (দুই মাথায় $O(1)$)
3. Standard lib-এর রত্ন: `Counter` (frequency এক লাইনে), `defaultdict` (grouping), `heapq` (top-K)
4. Comprehension idiomatic ও দ্রুত — কিন্তু বিশাল ডেটায় generator expression (`sum(x for ...)`) — list বানিয়ে মেমোরি না ভরে
5. dict Python 3.7+ এ insertion-ordered — interview-তে জিজ্ঞেস হয়; `OrderedDict` এখন প্রায় শুধু legacy
6. Mutable default argument (`def f(x, items=[])`) — ক্লাসিক ফাঁদ প্রশ্ন: default একবারই তৈরি হয়, সব call শেয়ার করে; `None` + ভেতরে তৈরি করা

৬. Type Hints ও আধুনিক Python Practices

Description: বড় টিমের Python-এ type hint এখন স্ট্যান্ডার্ড — কতদূর নেন এবং টুলিং কী, সেটার প্র্যাকটিক্যাল উত্তর।

৬টা Key Point:

1. Type hint runtime-এ কিছু করে না (annotation মাত্র) — চেক করে static tool: mypy/pyright; CI-তে চালালে তবেই দাম
2. কিন্তু Pydantic/FastAPI hint runtime-এ ব্যবহার করে — validation+docs hint থেকেই; Python-এ hint-এর সবচেয়ে বাস্তব ROI এটাই
3. আধুনিক syntax: `list[str], str | None` (3.10+) — `List, Optional` import-এর দিন শেষ
4. Dataclass vs Pydantic ভাগ: internal data holder → dataclass (হালকা); external/untrusted input → Pydantic (validate করে)
5. Tooling স্ট্যান্ডার্ড এক নিঃশ্বাসে: ruff (lint+format, black-এর উত্তরসূরি), mypy, pytest, uv/poetry (dependency) — "professional Python setup" প্রশ্নের উত্তর
6. Virtual env per project + lockfile — system Python-এ pip install করা টিম-অপরাধ; Docker-এ slim base + pinned versions

৭. Python-এ MongoDB ও Aggregation Pipeline

Description: Analytics/reporting সার্ভিসে Python + MongoDB জুটি — pipeline চিন্তা আর performance সচেতনতা (আমার দৈনন্দিন Sokrio-benchmark কাজ)।

৬টা Key Point:

1. Aggregation pipeline = ধাপের চেইন: `$match` → `$group` → `$project` → `$sort` — SQL-এর WHERE/GROUP BY-এর document-জগত রূপ
2. `$match` যত আগে তত ভালো + index-এর সাথে মিলিয়ে — pipeline-এর প্রথম stage index পেলে বাকিটা ছোট ডেটায় চলে
3. `$lookup` (join) সাবধানে — বড় collection-এ দামি; প্রায়ই ভালো সমাধান: ডেটা denormalize করে summary collection-এ রাখা
4. `$facet` দিয়ে এক query-তে একাধিক ফল (data + total count) — pagination-এ round trip বাঁচে
5. Python পাশে: pymongo (sync) / motor (async FastAPI-তে); cursor iterate করা — `list()` দিয়ে বিশাল ফল মেমোরিতে না তোলা
6. Multi-tenant প্যাটার্ন বলা: প্রতি tenant আলাদা DB/collection scope + cache key-তে tenant — এক tenant-এর রিপোর্ট আরেকজনে যেন না মেশে

৮. Testing Python: pytest Patterns

Description: pytest-এর idiomatic ব্যবহার — fixture, mock, parametrize; আর async code টেস্টের কৌশল।

৬টা Key Point:

1. pytest-এর শক্তি: plain assert + fixture DI — fixture-এ setup/teardown (yield-এর আগে setup, পরে cleanup)
2. Fixture scope বোঝা: function (default) vs session — দামি জিনিস (DB container) session-এ একবার
3. `@pytest.mark.parametrize` — এক টেস্ট, অনেক case; boundary case-গুলো টেবিল আকারে
4. Mock boundary-তে: external API/DB call patch (`monkeypatch/unittest.mock`) — যেখানে object ব্যবহৃত হয় সেখানে patch, যেখানে সংজ্ঞায়িত সেখানে নয় (ক্লাসিক ভুল)
5. FastAPI টেস্ট: `TestClient` (sync) বা `httpx.AsyncClient` — `dependency_overrides` দিয়ে auth/DB fake করা; পুরো অ্যাপ boot না করে route টেস্ট
6. টেস্ট দর্শন এক লাইনে: behavior টেস্ট, implementation নয় — refactor-এ টেস্ট সবুজ থাকা উচিত, নাহলে টেস্টই liability

৯. Python-এ Background Job ও Task Queue

Description: Celery-কেন্দ্রিক async processing — Laravel queue-র Python জগত; report generation/heavy কাজের প্যাটার্ন।

৬টা Key Point:

1. Celery = Python-এর queue স্ট্যান্ডার্ড: broker (Redis/RabbitMQ) + worker process + result backend — Laravel queue-র সরাসরি সমতুল্য
2. Task idempotent + ছোট payload (ID পাঠাও, object নয়) — দুই ecosystem-এ একই সোনার নিয়ম
3. Retry policy: `autoretry_for` + exponential backoff + `max_retries` — আর `acks_late=True` করলে crash-এ task হারায় না
4. Periodic কাজ: Celery Beat (বা সাধারণ cron/APScheduler ছোট ক্ষেত্রে) — nightly pre-aggregation এখানেই চলে
5. হালকা বিকল্প জানা: RQ (সরল), Dramatiq, বা FastAPI `BackgroundTasks` (একই process — শুধু ছোট, হারালে-ক্ষতি-নেই কাজে)
6. Monitoring: Flower dashboard / queue depth metric — আর ভারী রিপোর্টের progress DB-তে state রেখে ইউজারকে জানানো (queued → processing → done)

১০. Python vs PHP vs Node — Backend ভাষা বাছাই

Description: তিন স্ট্যাকে কাজ করা ইঞ্জিনিয়ারের কাছে প্রায় নিশ্চিত প্রশ্ন — কোন সার্ভিস কোন ভাষায় লিখবেন আর কেন।

৬টা Key Point:

1. Python জেতে: data/analytics/ML-ঘেঁষা সার্ভিস (pandas/numpy ecosystem), integration/scripting, aggregation-heavy reporting — লাইব্রেরির গভীরতাই কারণ
2. Laravel/PHP জেতে: business CRUD প্রোডাক্ট দ্রুত — full-stack convention, BD-তে টিম সহজলভ্য
3. Node জেতে: real-time + frontend-এর সাথে ভাষা ভাগাভাগি
4. Raw speed-এ Python ধীরতম — কিন্তু I/O-bound API-তে সেটা প্রায় অপ্রাসঙ্গিক; ভারী অংশ C-extension এ চলে; bottleneck প্রায় সবসময় DB
5. নিজের বাস্তব উদাহরণ দেওয়া: Sokrio-তে Laravel core (transactional) + Python/FastAPI benchmark সার্ভিস (multi-tenant analytics) — polyglot সিদ্ধান্তটা workload থেকেই এসেছে
6. পরিণত সমাপ্তি-লাইন: "ভাষা tool — টিম, ecosystem, আর সমস্যার আকৃতি মিলিয়ে বাছাই; ধর্মযুদ্ধ engineering না"